# Evidencia de Aprendizaje 1. Diseño Sistema de Información – Iteración 1

Juan David Henao Cárdenas
Jorge Eliécer Uribe Pulgarín
Santiago Andrés Valencia Duque

Institución Universitaria Digital de Antioquia IU Digital
Ingeniería de Software y Datos
Ingeniería de Software
Profesor: Reinaldo Rojas López
2026

Repositorio GitHub: https://github.com/andresvd95/IngSoftIUDig.git

## Introducción

El presente documento expone la planificación, especificación de requerimientos y modelado técnico correspondientes a la Iteración 1 del sistema de información para el parqueadero 'Autos Colombia'. Este proyecto se enfoca en resolver los desafíos operativos diarios asociados con el control manual, automatizando los procesos críticos para mitigar errores humanos y optimizar la experiencia del usuario. En esta primera fase, el alcance del software se delimita estrictamente a la gestión ágil y segura de la entrada y salida de vehículos que cuentan con el servicio de mensualidad activa.

A través de un enfoque de ingeniería de software estructurado, se definen los parámetros funcionales y no funcionales que guiarán el comportamiento del sistema. Asimismo, se incorporan las respectivas historias de usuario, diagramas de arquitectura y representaciones de la estructura de datos. Esta documentación técnica actúa como el puente formal entre las necesidades del negocio y el código fuente implementado, asegurando la alta disponibilidad, seguridad y rendimiento de la aplicación.

# Entregables del Sistema – Iteración 1

## 1. Requerimientos Funcionales y No Funcionales

A continuación, se especifican las restricciones y los comportamientos esperados del sistema de información enfocados en el control vehicular:

### Requerimientos Funcionales (RF):

• **RF-01: Registro de Entrada:** El sistema debe permitir al operador ingresar la placa de un vehículo, seleccionar el tipo (carro o moto) y registrar la hora de llegada automáticamente.
• **RF-02: Control de Celdas:** El sistema debe desplegar visualmente los puestos disponibles y cambiar su estado a 'Ocupado' al asignar un vehículo.
• **RF-03: Registro de Salida:** El sistema debe permitir la búsqueda por placa para asentar la fecha/hora de salida y liberar el espacio de parqueo.
• **RF-04: Lista de Vehículos Activos:** El sistema debe proveer un módulo para visualizar en tiempo real todos los automotores que permanecen dentro de las instalaciones.

### Requerimientos No Funcionales (RNF):

• **RNF-01: Seguridad y Autenticación:** El acceso a las operaciones del parqueadero está restringido a operarios autenticados mediante usuario y contraseña.
• **RNF-02: Tiempo de Respuesta:** El almacenamiento y la validación de registros en la base de datos no debe exceder los 3 segundos.
• **RNF-03: Robustez Local:** La aplicación debe contar con un despliegue capaz de operar localmente para asegurar la continuidad del servicio ante fallos de conectividad externa.

## 2. Historias de Usuario

### ID: HU-01 – Registrar Ingreso de Vehículo

**Como:** Operador del parqueadero.
**Quiero:** Digitar la placa de un vehículo que llega y asignarle un puesto vacío.
**Para:** Registrar la hora de entrada y actualizar el estado de ocupación de las celdas.
**Criterios de Aceptación:**
1. El sistema no debe permitir guardar el registro si el campo de la placa se encuentra vacío.
2. La interfaz solo debe mostrar para asignación aquellas celdas con estado 'Disponible'.
3. Una vez guardado el ingreso, la celda debe actualizar su estado a 'Ocupado' de forma inmediata.

**ID: HU-02 – Registrar Salida de Vehículo**

**Como:** Operador del parqueadero.

**Quiero:** Buscar la placa de un vehículo activo para marcar su salida.

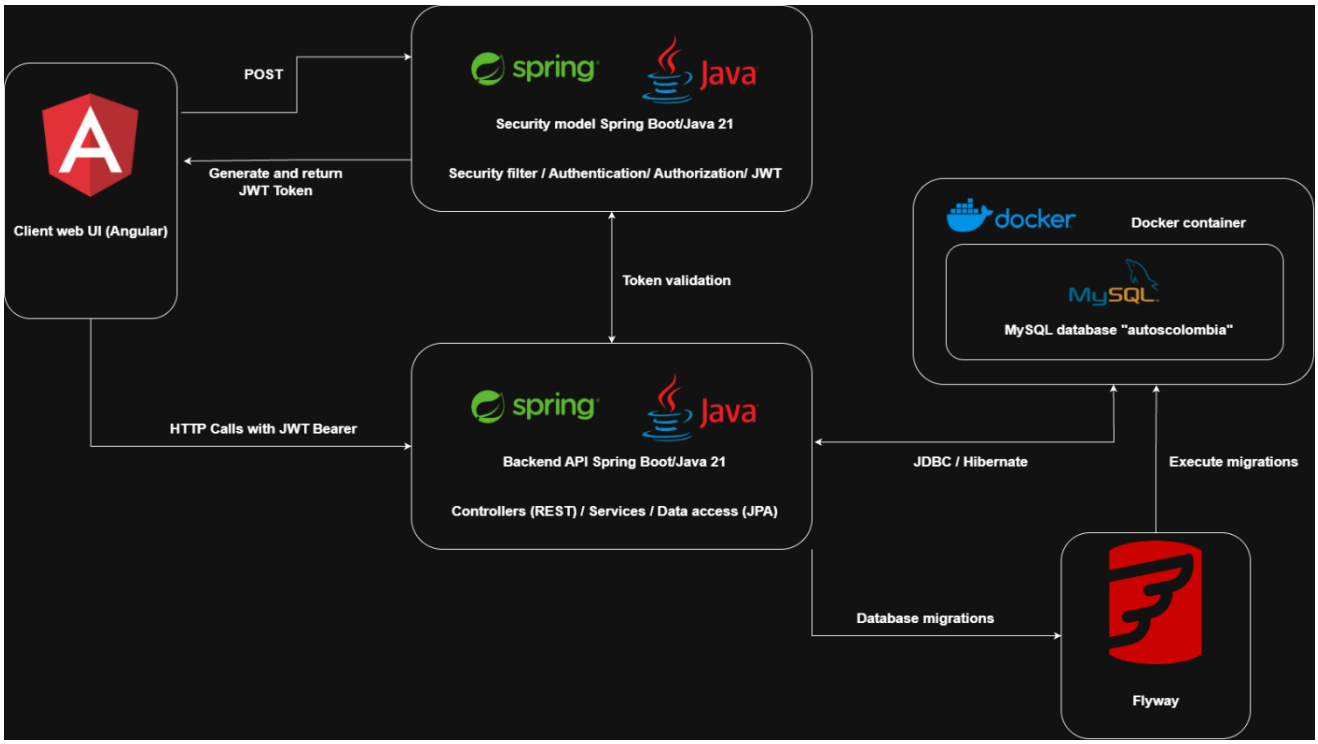
**Para:** Finalizar el registro de permanencia y liberar el espacio utilizado.

**Criterios de Aceptación:**

1. Al ingresar la placa, el sistema debe desplegar los datos de ingreso correspondientes.
2. Al confirmar la salida, el registro pasa al histórico y la celda asociada vuelve a estado 'Disponible'.

### 3. Arquitectura del Sistema

El diseño arquitectónico del software adopta un modelo desacoplado y escalable de tres capas, asegurando la separación de responsabilidades y facilitando el mantenimiento independiente de la interfaz y la lógica del servidor.



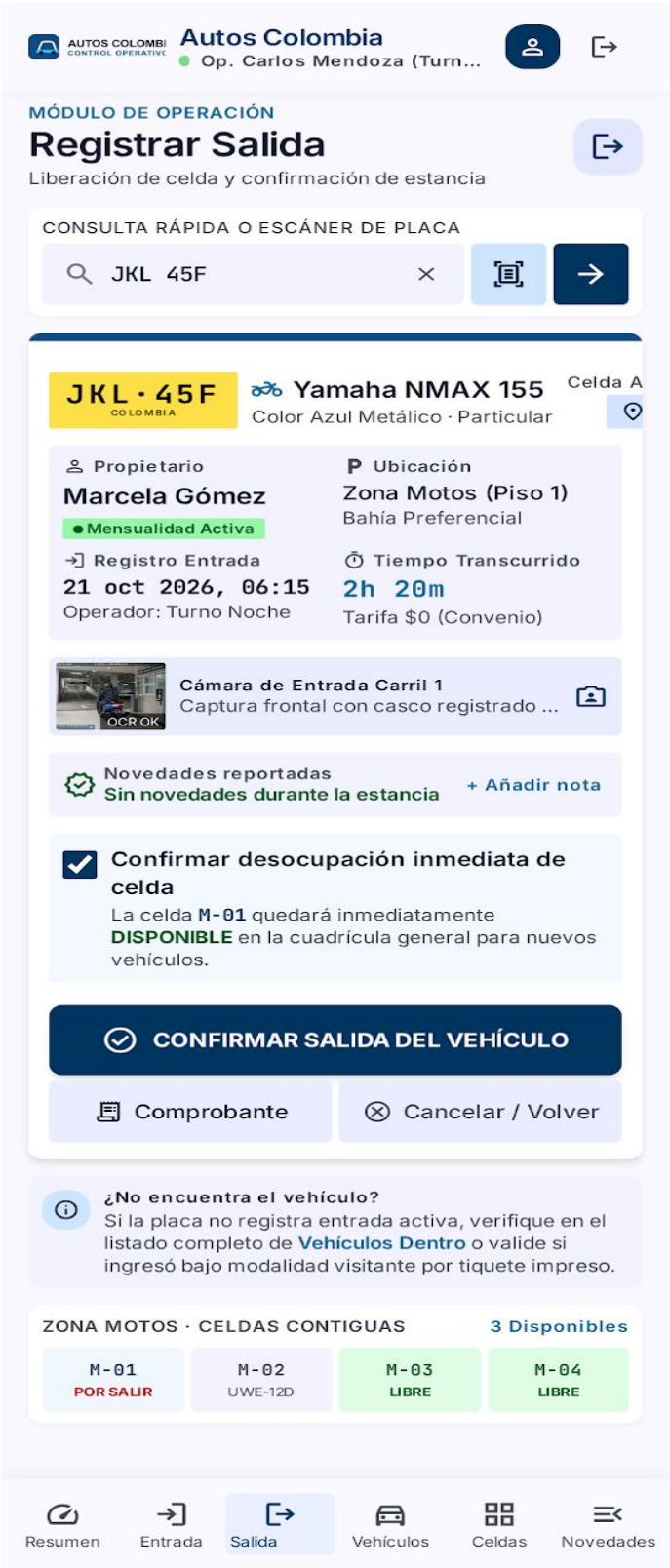

### 4. Diagramas de Casos de Uso, Clases y Paquetes

La interacción de los actores con el sistema, el mapeo de los componentes lógicos de la aplicación y la organización de los módulos se evidencian a través de los prototipos funcionales de la interfaz de usuario en la presente iteración:

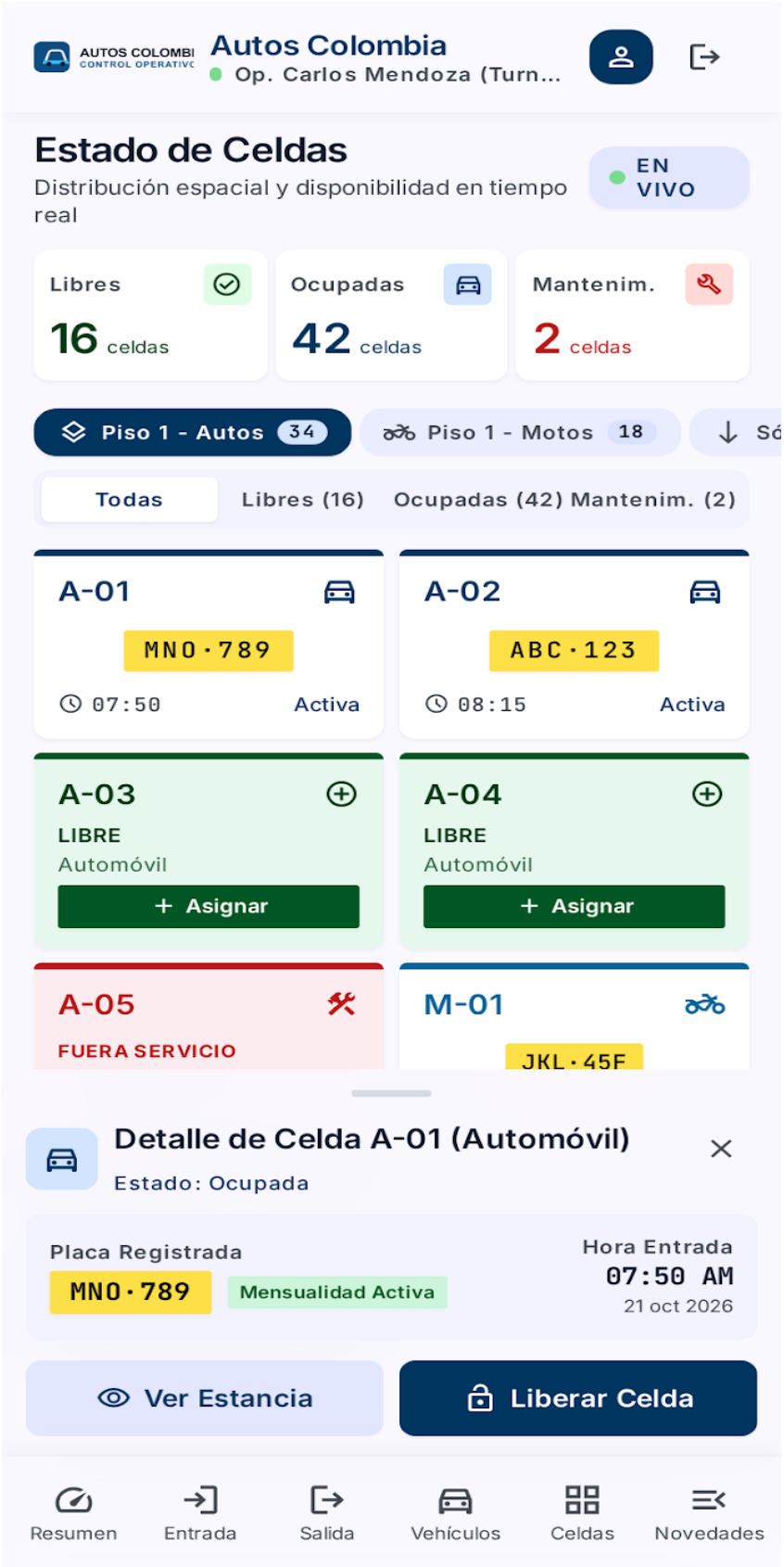
## Evidencia de Casos de Uso (Módulo Entrada)



**Evidencia de Diagrama de Clases (Módulo Salida)**



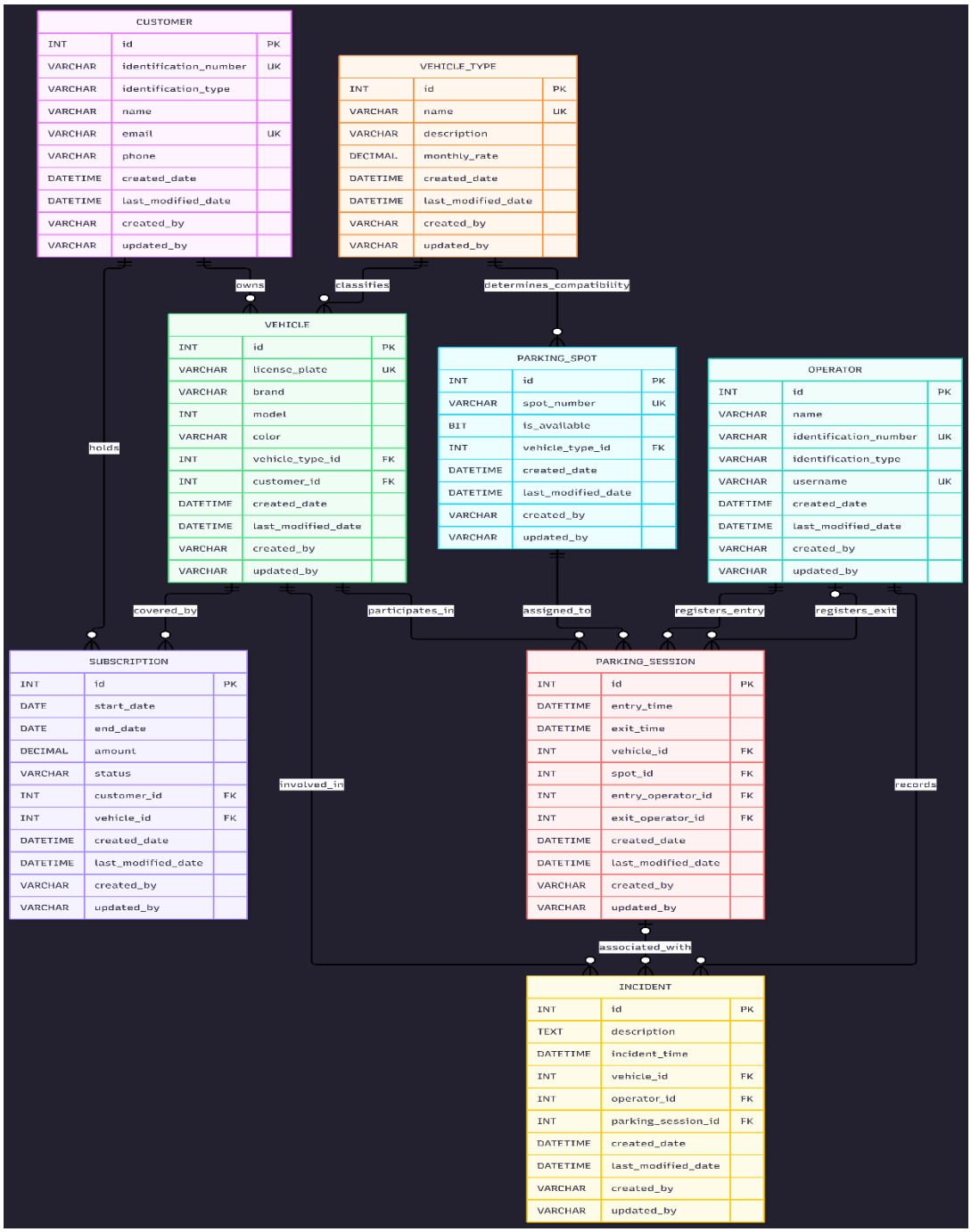
**Evidencia de Diagrama de Paquetes (Módulo Celdas)**



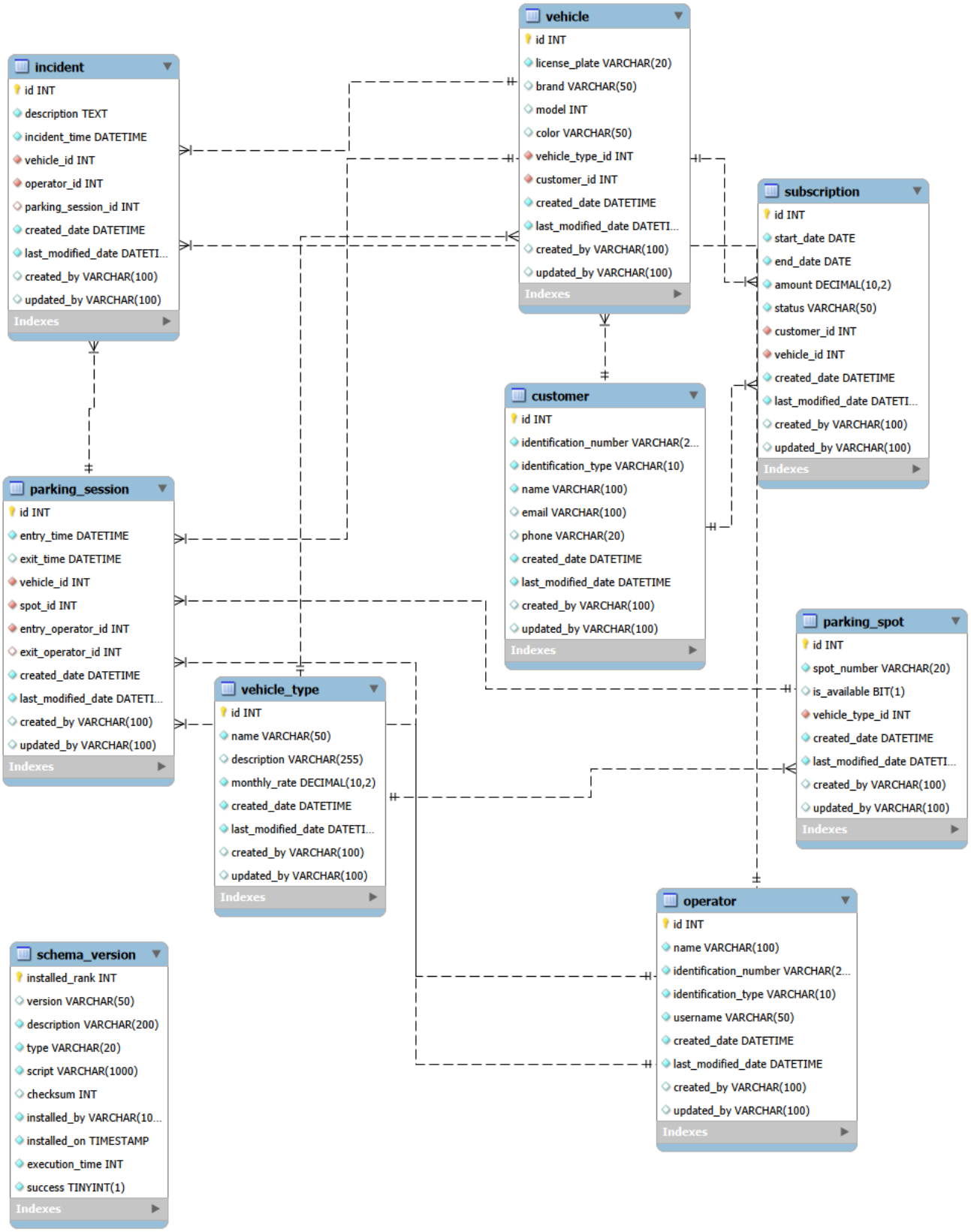
## 5. Modelo Entidad-Relación (MER), Relacional (MR) y Físico (MF)

La estructura de persistencia de datos ha sido diseñada de forma relacional y normalizada para garantizar la consistencia en el almacenamiento de usuarios, vehículos, celdas y el registro de estancias concurrentes.

### Modelo Entidad-Relación (MER)



## Modelo Relacional y Físico (MR / MF)



## Conclusiones

**Primera:** La implementación de los requerimientos e historias de usuario definidos para la Iteración 1 automatiza con éxito el núcleo operativo de 'Autos Colombia'. Al digitalizar el control de entrada y salida, se reduce drásticamente el riesgo de errores en registros manuales y se agiliza la atención al cliente con mensualidad vigente.

**Segunda:** El modelado arquitectónico en tres capas y el diseño normalizado de la base de datos aseguran que el sistema sea robusto, seguro y altamente escalable. La coherencia entre los diagramas técnicos y el código fuente garantiza una estructura preparada para incorporar sin traumatismos las siguientes iteraciones de gestión de pagos y novedades.